

Using unknown pointers and references in constant expressions

Document #: P2280R2
Date: 2021-05-15
Project: Programming Language C++
Audience: EWG
Reply-to: Barry Revzin
<barry.revzin@gmail.com>

Contents

[1 Revision History](#)

[2 Introduction](#)

[2.1 Wait, why?](#)

[2.2 Other Examples](#)

[2.3 The `this` pointer](#)

[2.4 Other pointers](#)

[3 Proposal](#)

[3.1 Implementation Experience](#)

[3.2 Other not-quite-reference examples](#)

[3.3 Lifetime Dilemma](#)

[3.4 Still further cases](#)

[3.5 What about `nullptr`?](#)

[3.6 Wording](#)

[4 Acknowledgments](#)

[5 References](#)

§ 1 Revision History

[[P2280R1](#)] extended R0 to also include `this`. This revision extends that further to consider pointers-to-unknown in addition to references-to-unknown.

[[P2280R0](#)] was discussed at the EWG telecon on Feb 3, 2021. The following polls were taken:

The use cases presented in P2280 are problems in C++’s specification of `constexpr`, and we would like to fix these problems, ideally in C++23.

SF	F	N	A	SA
3	14	2	0	0

This should be a Defect Report against C++20, C++17, C++14, and C++11.

SF	F	N	A	SA
3	11	4	0	0

Send P2280 to Electronic Polling, with the intent of going to Core, after getting input from MSVC and GCC implementors.

SF	F	N	A	SA
8	10	1	0	0

This revision updates wording. This revision also adds discussion of [the this pointer](#), and extends the proposal to additional cover [this](#) (but not arbitrary pointers)

§ 2 Introduction

Let’s say I have an array and want to get its size as a constant expression. In C, I had to write a macro:

```
#define ARRAY_SIZE(a) (sizeof(a)/sizeof(a[0]))
```

But in C++, we should be able to do better. We have `constexpr` and templates, so we can use them:

```
template <typename T, size_t N>
constexpr auto array_size(T (&)[N]) -> size_t {
    return N;
}
```

This seems like it should be a substantial improvement, yet it has surprising limitations:

```
void check(int const (&param)[3]) {
    int local[] = {1, 2, 3};
    constexpr auto s0 = array_size(local); // ok
    constexpr auto s1 = array_size(param); // error
}
```

The goal of this paper is to make that second case, and others like it, valid.

§ 2.1 Wait, why?

The reason is that in order for `array_size(param)` to work, we have to pass that reference to `param` into `array_size` - and that involves “reading” the reference. The specific rule we’re violating is 7.7 [\[expr.const\]/5.12](#):

⁵ An expression *E* is a *core constant expression* unless the evaluation of *E*, following the rules of the abstract machine ([intro.execution]), would evaluate one of the following:

(5.12) — an *id-expression* that refers to a variable or data member of reference type unless the reference has a preceding initialization and either

(5.12.1) — it is usable in constant expressions or

(5.12.2) — its lifetime began within the evaluation of *E*;

The reason we violate the reference rule is due to the underlying principle that the constant evaluator has to reject all undefined behavior, so the compiler has to check that all references are valid.

This would be more obvious if our situation used pointers instead of references:

```
template <typename T, size_t N>
constexpr size_t array_size(T (*)(N)) {
    return N;
}

void check(int const (*param)[3]) {
    constexpr auto s2 = array_size(param); // error
}
```

This case is perhaps more clear as to why it's ill-formed: copying a function parameter during constant evaluation means having to read it in order to copy it. It has to itself be a constant expression, and function parameters are not constant expressions - even in `constexpr` or `constexpr` functions.

But if the `param` case is ill-formed, why does the `local` case work? An unsatisfying answer is that... there just isn't any rule in `[expr.const]` that we're violating. There's no lvalue-to-rvalue conversion (we're not reading through the reference in any way yet) and we're not referring to a reference (that's the previous rule we ran afoul of). With the `param` case, the compiler cannot know whether the reference is valid, so it must reject. With the `local` case, the compiler can see for sure that the reference to `local` would be a valid reference, so it's happy.

Notably, the rule we're violating is only about *references*. We can't write a function that takes an array by value, so let's use the next-best thing: `std::array` and use the standard library's `std::size` (cppref):

```
void check_arr_val(std::array<int, 3> const param) {
    std::array<int, 3> local = {1, 2, 3};
    constexpr auto s3 = std::size(local); // ok
    constexpr auto s4 = std::size(param); // ok
}
```

If `param` were a reference, the initialization of `s4` would be ill-formed (for the same reason as previously), but because it's a value, this is totally fine.

So as long as you pass all your containers around by value, you're able to use `get` and use the `size` as a constant expression. Which is the kind of thing that's intellectually interesting, but also wildly impractical because obviously nobody's about to start passing all their containers around *by value*.

§ 2.2 Other Examples

Here are few other cases, which currently are ill-formed because of this reference-to-unknown rule.

From Andrzej Krzemienski:

Another situation where being able to use a reference to a non-core-constant object is when I am only interested in the type of the reference rather than the value of the object:

```
template <typename T, typename U>
constexpr bool is_type(U &&)
{
    return std::is_same_v<T, std::decay_t<U>>;
}
```

So that I can use it like this:

```
auto visitor = [](auto&& v) {
    if constexpr(is_type<Alternative1>(v)) {
        // ...
    } else if constexpr(is_type<Alternative2>(v)) {
        // ...
    }
};
```

I can do it with a macro:

```
#define IS_TYPE(TYPE, EXPR) (std::is_same_v<TYPE,
    std::decay_t<decltype(EXPR)>>)
```

From Jonathan Wakely:

```
auto rando(std::uniform_random_bit_generator auto& g)
{
    if constexpr (std::has_single_bit(g.max() - g.min()))
        // ...
    else
        // ...
}
```

The concept requires that `g.max()` and `g.min()` are `constexpr` static member functions, so this should work. And if I did it with an object of that type, it would work. But because `g` is a reference, it's not usable in a constant expression. That makes it awkward to refactor code into a function (or function template), because what worked on the object itself doesn't work in a function that binds a reference to that object.

I can rewrite it as something like:

```
using G = remove_reference_t<decltype(g)>;
if constexpr (std::has_single_bit(G::max() - G::min()))
```

Or avoid abbreviated function syntax so I have a name for the type:

```
template<std::uniform_random_bit_generator G>
auto rando(G& g)
{
    if constexpr (std::has_single_bit(G::max() - G::min()))
    }
```

But it's awkward that the first version doesn't Just Work.

Another from me:

I have a project that has a structure like:

```
template <typename... Types>
struct Widget {
    struct Config : Types::config... {
        template <typename T>
        static constexpr auto sends(T) -> bool {
            return std::is_base_of_v<typename T::config, Config>;
        }
    };

    Config config;
};
```

With the intent that this function makes for a nice and readable way of doing dispatch:

```
void do_configuration(auto& config) {
    // the actual type of config is... complicated

    if constexpr (config.sends(Goomba{})) {
        // do something
    }
    if constexpr (config.sends(Paratroopa{})) {
        // do something else
    }
}
```

Except this doesn't work, and I have to write:

```
void do_configuration(auto& config) {
    using Config = std::remove_cvref_t<decltype(config)>;

    if constexpr (Config::sends(Goomba{})) {
        // ...
    }
}
```

Which is not really “better.”

What all of these examples have in common is that they are using a reference to an object of type `T` but do not care at all about the identity of that object. We're either querying properties of the type, invoking static member functions, or even when invoking a non-static member function (as in `std::array::size`), not actually accessing any non-static data members. The result would be the same for every object of type `T`... so if the identity doesn't change the result, why does the lack of identity cause the result to be non-constant? It's very much constant.

§ 2.3 The `this` pointer

Consider the following example, very similar to one I shared earlier. Here, we need to read a constant through a member, so we write our member function two different ways (the latter using [P0847R6](#)):

Regular non-static member function	With deducing this
<pre>template <bool V> struct Widget { struct Config { static constexpr bool value = V; } config; void f() { if constexpr (config.value) { // ... } } };</pre>	<pre>template <bool V> struct Widget { struct Config { static constexpr bool value = V; } config; void f(this Widget& self) { if constexpr (self.config.value) // ... } };</pre>

Even if we drop the restriction on using references-to-unknown (the extent of the R0 proposal of this paper), the example on the left is still ill-formed. Because we don't even have a reference here exactly, we're accessing through `this`, and one of the things we're not allowed to evaluate as part of constant evaluation is the first bullet from [expr.const]/5:

(5.1) — `this`, except in a constexpr function that is being evaluated as part of E;

And here, `Widget<V>::f` is not a `constexpr` function.

However, the example on the right is valid with the suggested rule change. Here, `self` is a reference-to-unknown and `value` ends up being a `constexpr` variable that we can read. So this works. This example wasn't exactly what we had in mind when we wrote that paper though, and while we would be happy to keep dumping motivating use-cases into that paper... it doesn't exactly seem like a meaningful solution to the problem. It seems pretty unsatisfactory that `self.config.value` is okay while `(*this).config.value` is not, when `self` and `(*this)` mean the same thing in this context.

So that's also fairly unsatisfying. It would be nice to simply support this use-case as well. `this`, after all, is a reference (practically speaking).

§ 2.4 Other pointers

The thing is though: why just the `this` pointer and not all pointers? For that matter, is there really a meaningful distinction between pointers and references?

Is there a meaningful distinction between supporting these examples?

References	Pointers
<pre>template <typename T, size_t N> constexpr auto array_size(T (&)[N]) -> size_t { return N; }</pre>	<pre>template <typename T, size_t N> constexpr auto array_size(T (*)[N]) -> size_t { return N; }</pre>

References	Pointers
<pre>void check(int const (&param)[3]) { constexpr auto s = array_size(param); }</pre>	<pre>void check(int const (*param)[3]) { constexpr auto s = array_size(param); }</pre>

Pointers require a lot more specification effort, since pointers allow more operations, and we'd have to define what all of those things mean. For instance:

```
void f(std::array<int, 3>& r, std::array<int, 4>* p) {
    static_assert(r.size() == 3);    // #1
    static_assert(p->size() == 4);   // #2
    static_assert(p[3].size() == 4); // #3
    static_assert(&r == &r);         // #4
}
```

#1 is one of the motivating examples in the paper. #2 would require dereferencing a pointer, which is similar to accessing through a reference yet isn't exactly the same. #3 additionally requires array access and we have no idea if p actually points to an array, much less what the size of that array would be. But both #2 and #3 generally fit the notion that these are expressions that either have a particular constant value or are undefined behavior, although #2 only requires that p be a pointer to unknown object while #3 requires p be a pointer to an unknown array of objects.

#4 is interesting in a different way: here this actually has to be true, but in order support that, rather than simply tracking that &r is "pointer to known array<int, 3>", we have to additionally track that it is specifically a pointer to r. This, at least in EDG, is a much bigger change (with much less commensurate value).

The problem is, while changing the specification to support #1 is largely around *not* rejecting the case, supporting #2 is a much more involved process. We not only have to introduce the concept of pointer-to-unknown but we also have to specify what all the operations mean. We have to say what a pointer-to-unknown means. That it dereferences into a reference-to-unknown and likewise that taking the address of a reference-to-unknown yields a pointer-to-unknown.

But then we also have to define what the various other operations on pointers to references are. What about addition and subtraction and indexing (i.e. #3)? Equality (i.e. #4)? Ordering? If we reject indexing, what about p[0]?

Supporting references-to-unknown is largely about *not* rejecting those cases that are currently rejected. Similarly, supporting `this` in the context of (implicit or explicit) class member access is likewise simply about not rejecting. In order to support pointers-to-unknown, we likewise try to push rejecting cases as far as possible. That is, indirecting through a τ^* with unknown value just gives you some unknown τ .

But what about the other operations? Comparing two pointers, where at least one is a pointer-to-unknown, cannot be a constant expression so will have to be rejected. There is a notable exception here in doing something like `p == p` which could potentially be `true` but seems exceedingly narrow. What about pointer arithmetic? Should the #3 example above work or not? Would your answer change if instead of a

pointer we had an array of unknown bound (there’s an example of such later in this paper)? What if it were `p[0]` instead of `p[3]`?

This paper takes a very narrow position here: indirecting through a τ^* with unknown value is fine, but that’s all you can do with it. That is, pointers-to-unknown behave a lot like references-to-unknown that are just spelled differently. No pointer arithmetic, comparison, invocation, etc.

§ 3 Proposal

The proposal is to allow all these cases to just work. That is, if during constant evaluation, we run into a reference with unknown origin, this is still okay, we keep going. Similarly, if we run into a pointer with unknown origin, we allow indirecting through it.

Some operations are allowed to propagate a reference-to-unknown or pointer-to-unknown node (such as class member access or derived-to-non-virtual-base conversions). But most operations are definitely non-constant (such as lvalue-to-rvalue conversion, assignment, any polymorphic operations, conversion to a virtual base class, etc.). This paper is *just* proposing allowing those cases that work irrespective of the value of the reference or pointer (i.e. those that are truly constant), so any operation that depends on the value in any way needs to continue to be forbidden.

Notably, this paper is definitively *not* proposing any kind of short-circuiting evaluation. For example:

```
constexpr auto g() -> std::array<int, 10>&;
static_assert(g().size() == 10);
```

This check still must evaluate `g()`, which may or may not be a constant expression in its own right, even if `g().size()` is “obviously” 10. This paper is focused solely on those cases where we have an *id-expression* of reference or pointer type.

§ 3.1 Implementation Experience

I’ve implemented this in EDG at least to the extent that the test cases presented in this paper all pass, whereas previously they had all failed.

§ 3.2 Other not-quite-reference examples

There are a few other closely related examples to consider for how to word this proposal. All of these are courtesy of Richard Smith.

We generally assume the following works:

```
auto f() {
    const int n = 5;
    return [] { int arr[n]; };
}
```

but `n` might not be in its lifetime when it’s read in the evaluation of `arr`’s array bound. So we need to add wording to actually make that work.

Then there are further lifetime questions. The following example is similar to the other examples presented earlier:

```
struct A { constexpr int f() { return 0; } };
struct B : A {};
void f(B &b) { constexpr int k = b.f(); }
```

But this one is a bit different:

```
struct A2 { constexpr int f() { return 0; } };
struct B2 : virtual A2 {};
void f2(B2 &b) { constexpr int k = b.f(); }
```

Here, we convert `&b` to `A2*` and that might be undefined behavior (as per [class.ctor]/3). But this case seems similar enough to the earlier cases and should be allowed: `b.f()` is a constant, even with a virtual base. We need to ensure then that we consider references as within their lifetimes.

§ 3.3 Lifetime Dilemma

If we go back to this example:

```
extern B2 &b;
constexpr int k = b.f();
```

It seems reasonable to allow it, having no idea what the definition of `b` is. But what if we *do* see the definition of `b`, and it's:

```
union U { char c; B2 b2; };
constexpr U u = { .c = 0 };
B2 &b = const_cast<B2*>(u.b2);
```

Now we *know* `b` isn't within its lifetime. We added more information, and turned our constant expression into a non-constant expression?

However, there's a reasonable principle here: anything that has only one possible interpretation *with defined behavior* has that defined behavior for constant evaluation purposes. This is true of all the examples presented up until now.

§ 3.4 Still further cases

A different case is the following:

```
struct A { virtual constexpr int f() { return 0; } } a;
constexpr int k = a.f();
constexpr auto &ti = typeid(a);
constexpr void *p = dynamic_cast<void*>(&a);
```

Here, `A::f` is `virtual`. Which might make it seem constant, but any number of shenanigans could ensue — like placement-new-ing a derived type (of the same size) over `a`. So all of these should probably remain non-constant expressions.

Perhaps the most fun example is this one:

```
extern const int arr[];
constexpr const int *p = arr + N;
constexpr int arr[2] = {0, 1};
constexpr int k = *p;
```

Which every compiler currently provides different results (in order of most reasonable to least reasonable):

1. Clang says `arr+N` is non-constant if `N != 0`, and accepts with `N == 0`.
2. GCC says `arr+N` is always constant (even though it sometimes has UB), but rejects reading `*p` if `arr+N` is out of bounds.
3. ICC says `arr+N` is always constant (even though it sometimes has UB), but always rejects reading `*p` even if `arr+N` is in-bounds.
4. MSVC says you can't declare `arr` as non-constexpr and define it constexpr, even though there is no such rule

This, to me, seems like there should be an added rule in `[expr.const]` that rejects addition and subtraction to an array of unknown bound unless that value is 0. This case seems unrelated enough to the rest of the paper that I think it should just be a Core issue.

§ 3.5 What about `nullptr`?

Consider this example from David Stone:

```
template <typename T, int N>
struct array {
    constexpr auto size() const -> int { return N; }
};

constexpr void a(array<int, 1>* p) {
    constexpr int x = p->size();
}

constexpr bool f() {
    a(nullptr);
    return true;
}

static_assert(f());

int b(array<int, 1>* p) {
    return p->size();
}

int main() {
    return b(nullptr);
}
```

Here we have two pieces of code that each try to invoke a member function on a null pointer: one as a constant expression and one at runtime. With this paper as worded, the former may be well-defined (declaring `x` begins a new constant evaluation in which `p` has already started its lifetime and is not itself a constant expression, and so we have a pointer-to-unknown... which we can then invoke a non-static member on just fine because at no point do we read through the pointer or do anything else that involves knowing anything about the actual array object). But the latter would still be undefined behavior (invoking a non-static member function on a null pointer). This presents a fairly odd and unsatisfying situation where we have some code that is undefined behavior at runtime but... well-defined at compile time? That's the opposite of the way this usually works!

This kind of example suggests four possible directions:

1. Go back to [P2280R0] which only suggested allowing references to unknown, which sidesteps this question entirely.
2. Go back to [P2280R1] which only suggested allowing references to unknown and `this`, which *mostly* sidesteps this question in a way that's much more palatable since by the time you enter a member function and `this` would be used, we're kind of in a different situations where this question just isn't in your face.
3. Accept the situation proposed in this paper, where we introduce this potential duality in scenarios like this.
4. Go further in the other direction and actually make David Stone's example well-formed at runtime. That is, define invoking a member function on a null pointer as being okay as long as you don't actually have to read any non-static data members (i.e. the same conditions that would other prevent equivalent code from being a constant expression). In other words, we widen both the allowed code during constant evaluation time and also the allowed code during runtime.

This paper is currently worded going in the direction of (3), but based on the EWG Telecon on May 6th, 2021, it may be more palatable to go in one of the other directions. Note that even option (1) isn't necessarily a panacea here since I can rewrite David Stone's example as:

```
constexpr void a(array<int, 1>* p) {  
    array<int, 1>& r = *p;  
    constexpr int x = r.size();  
}
```

Is this really that different? Here, the cause of the undefined behavior is in a slightly different place: dereferencing the potentially-null pointer rather than directly invoking a function on it. But the end result is kind of the same: R0 of this paper proposes this to be well-formed, and if we adopt either direction (1) or (2) then users would have to work around using pointers by introducing named references. Which doesn't seem like it makes the code better.

§ 3.6 Wording

We need to strike the 7.7 [expr.const]/5.12 rule that disallows using references-to-unknown during constant evaluation and the 5.1 rule that disallows using `this` outside of `constexpr` functions, and add

new rules to reject polymorphic operations on unknown objects and rejecting various pointer-to-unknown operations:

5 An expression *E* is a *core constant expression* unless the evaluation of *E*, following the rules of the abstract machine ([intro.execution]), would evaluate one of the following:

- (5.1) — ~~this, except in a constexpr function that is being evaluated as part of E;~~
- (5.1) — an operation which has an operand that is an expression of pointer type that points to an unspecified object within the evaluation of *E*, if that operation is one of the following:
 - (5.1.1) — addition or subtraction ([expr.add]),
 - (5.1.2) — comparison ([expr.eq], [expr.rel]),
 - (5.1.3) — increment or decrement ([expr.pre.incr]), or
 - (5.1.4) — boolean conversion ([conv.bool]),
- (5.2) — [...]
- (5.5) — an invocation of a virtual function for an object unless the object's dynamic type is known and either
 - (5.5.1) — the object is usable in constant expressions or
 - (5.5.2) — its lifetime began within the evaluation of *E*;
- (5.7) — [...]
- (5.8) — an lvalue-to-rvalue conversion unless it is applied to
 - (5.8.1) — a non-volatile glvalue that refers to either an object that is usable in constant expressions or an object of pointer type, or
 - (5.8.2) — a non-volatile glvalue of literal type that refers to a non-volatile object whose lifetime began within the evaluation of *E*
- (5.9) — [...]
- (5.10) — [...]
- (5.11) — an invocation of an implicitly-defined copy/move constructor or copy/move assignment operator for a union whose active member (if any) is mutable, unless the lifetime of the union object began within the evaluation of *E*;
- (5.12) — ~~an id-expression that refers to a variable or data member of reference type unless the reference has a preceding initialization and either~~
 - (5.12.1) — ~~it is usable in constant expressions or~~
 - (5.12.2) — ~~its lifetime began within the evaluation of E;~~
- (5.13) — in a *lambda-expression*, a reference to `this` or to a variable with automatic storage duration defined outside that *lambda-expression*, where the reference would be an odr-use;
- (5.14) — [...]


```

static_assert(&swam == &swam);           // error: performing a
    comparison operation involving       // a pointer to unspecified
    object

constexpr int a = (&swam + 0)->phelps(); // error: performing an
    addition on a pointer to           // unspecified object
constexpr Swim* swim = &swam;           // error: a pointer to
    unspecified object is not a        // permitted result of a
    constant expression
}

extern Swim dc;
extern Swim& trident;

constexpr auto& x = typeid(dc);          // ok: can only be typeid(Swim)
constexpr auto& y = typeid(trident);     // error: unknown dynamic type

— end example]

```

Add a note to [expr.const]/11 to make it clear that these are not permitted results:

- ¹¹ An entity is a *permitted result of a constant expression* if it is an object with static storage duration that either is not a temporary object or is a temporary object whose value satisfies the above constraints, or if it is a non-immediate function. [*Note: A glvalue core constant expression that either refers to or points to an unspecified object is not a constant expression.* - end note]

§ 4 Acknowledgments

Thanks to Daveed Vandevoorde for the encouragement and help. Thanks to Richard Smith for carefully describing the correct rule on the reflector and helping provide further examples and wording. Thanks to Michael Park for pointing out the issue to me, Tim Song for explaining it, and Jonathan Wakely for suggesting I pursue it.

§ 5 References

[P0847R6] Barry Revzin, Gašper Ažman, Sy Brand, Ben Deane. 2021-01-15. Deducing this.

<https://wg21.link/p0847r6>

[P2280R0] Barry Revzin. 2021-01-13. Using unknown references in constant expressions.

<https://wg21.link/p2280r0>

[P2280R1] Barry Revzin. 2021-02-15. Using unknown references in constant expressions.

<https://wg21.link/p2280r1>